Mini Distributed File Storage System – Project Report

Oct 28, 2026 · @Nestor

Abstract

We built a working distributed file storage system in Python and Flask that keeps every file available when a storage node fails. One master node holds the chunk map, and three storage nodes hold the data as 1 MB chunks, each stored on two different nodes.

All seven automated tests passed. With one node stopped, a 5 MB file downloaded complete in 0.05 s with an identical SHA-256 checksum, and the master restored two copies of every chunk 10.0 s after the crash. With two of the three nodes down the file still downloaded, and 10 files of different sizes spread exactly 30 / 30 / 30 chunk copies across the nodes.

Keywords: distributed file system, HDFS, replication, fault tolerance, heartbeat, self-healing, data integrity.

1. Introduction and problem statement

A single server cannot store big data safely: it runs out of space, and when it fails every file on it becomes unreachable. This project shows, at small scale, how a distributed file system removes both limits by spreading each file across several machines and keeping more than one copy of every piece.

The design follows the Hadoop Distributed File System (HDFS). A file is cut into fixed-size chunks, each chunk is stored on two different storage nodes, and a master node keeps the map of where every chunk lives. When a storage node stops, the client reads the surviving copy, and the master copies the lost chunks to a healthy node so that two copies exist again.

Problem statement. Big data files are too large for one machine, and machines fail. A single server is both a storage limit and a single point of failure. The question this project answers is: can a file still be downloaded, complete and unchanged, after one of the machines that stored it has stopped?

2. Objectives and scope

The project set five objectives, and each one is checked by at least one test in Section 7.

1. **Partition data:** split every file into 1 MB chunks and spread the chunks across three storage nodes.
2. **Replicate:** store every chunk on two different nodes (replication factor 2).
3. **Tolerate failure:** keep every file downloadable, complete and unchanged, while a storage node is down.
4. **Detect and heal:** notice a failed node automatically and re-create its lost copies on a healthy node.
5. **Measure:** time uploads and downloads, and chart how chunks spread across the nodes.

In scope: one master node, three storage nodes and a command-line client, all on one laptop (ports 5000 to 5003), with 1 MB chunks and a replication factor of 2.

Out of scope: user accounts, a web front end, encryption and deployment across several machines. Leaving these out kept the 10-day project focused on the distributed-systems ideas the course examines.

3. Background concepts

Seven distributed-systems concepts carry the design; each maps to a concrete part of the code.

Concept	Meaning	Where it appears in this project
Data partitioning	A large file is cut into pieces that live on different machines	Files are split into 1 MB chunks spread across 3 nodes
Replication	Each piece is kept on more than one machine	Every chunk is stored on 2 different nodes
Fault tolerance	The system keeps working when a part fails	Downloads succeed with a node down
Failure detection	Machines report regularly; silence means failure	Heartbeat every 3 s; dead after 10 s of silence
Self-healing	Lost copies are re-created automatically	The master orders a surviving node to copy each lost chunk
Master–worker architecture	One coordinator holds the map; workers hold the data	Master keeps metadata; storage nodes keep chunks
Data integrity	Data can be proven unchanged	SHA-256 per chunk and per file, checked on every read

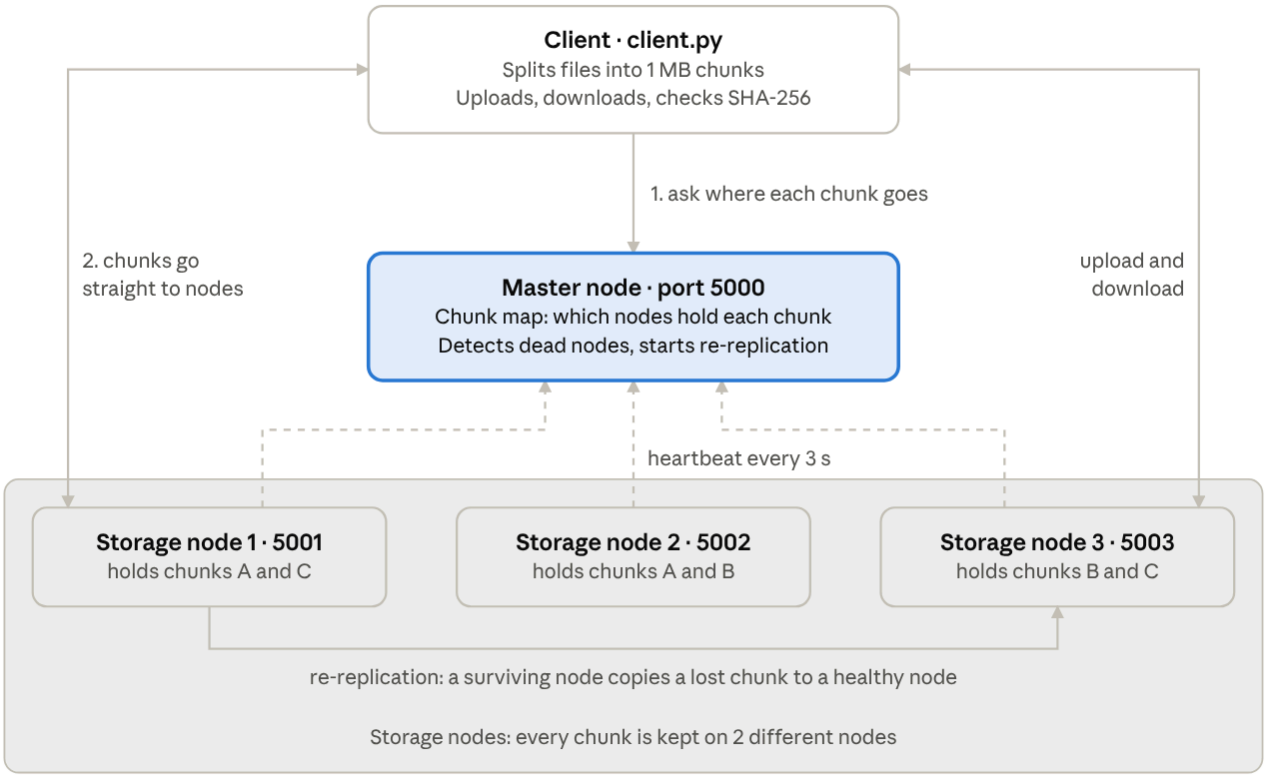
How this maps to HDFS. HDFS has one NameNode that stores all file-system metadata and many DataNodes that store blocks (Apache HDFS design). Our master plays the NameNode and our storage nodes play the DataNodes. The parameters are scaled down so that the behaviour shows in a classroom demo.

Feature	HDFS default	This project
Block (chunk) size	128 MB	1 MB
Replication factor	3	2
Heartbeat interval	3 s	3 s
Node declared dead after	over 10 minutes	10 s
Checksums	per block, checked on read	SHA-256 per chunk and per file
Writes	write once, read many	write once; replace to change

HDFS figures are from the HDFS design guide and hdfs-default.xml. HDFS waits over 10 minutes on purpose, to avoid copying storms when a node is only briefly slow; our 10 s keeps the demo short.

4. System design and architecture

The system has three kinds of component: one master that holds only the chunk map, three storage nodes that hold the chunks, and a client that moves the data. All run on one laptop on ports 5000 to 5003 and talk over HTTP.



system architecture · 1 master, 3 storage nodes, 1 client

In this example a 3-chunk file survives the loss of any one node, because every chunk still has a copy on another node.

Components

- **Master node (port 5000)** stores, for every file, its size, its SHA-256 and the list of nodes holding each chunk. It tracks which nodes are alive from their heartbeats and starts re-replication when one dies. It never stores or forwards file data.
- **Storage nodes (ports 5001 to 5003)** each keep chunks in their own folder (`storage/node_5001/` and so on). A node does not know which file a chunk belongs to; it only saves, returns, lists, deletes and copies chunks.
- **Client** splits files into 1 MB chunks, uploads them, downloads and rebuilds files, and checks checksums.

Upload flow

1. The client reads the file size and works out the number of 1 MB chunks.
2. It asks the master where each chunk should go (`POST /allocate`); the master picks the 2 least-loaded live nodes for each chunk.
3. The client sends each chunk, with its SHA-256 in a header, straight to both nodes (`PUT /chunks/<id>`); each node verifies the checksum before saving.

4. The client reports the stored chunks, the file size and the file's SHA-256 (`POST /commit`), and the master saves the chunk map.

Download flow

1. The client asks the master for the chunk map (`GET /files/<name>`), which marks the copies on live nodes.
2. For each chunk, the client reads the first live copy and checks the chunk's SHA-256; if a node is down or the chunk is damaged, it reads the other copy.
3. The client joins the chunks in order, checks the whole file's SHA-256 against the original, and only then saves the file.

The master's metadata (from `metadata.json` , after uploading the demo file):

```
{
  "reviews.csv": {
    "size": 3145659,
    "sha256": "33206b0bcd9a...",
    "uploaded": "2026-10-28 11:37:07",
    "chunks": [
      {"id": "reviews.csv_0", "nodes": [5001, 5002], "size": 1048576,
"sha256": "..."},
      {"id": "reviews.csv_1", "nodes": [5003, 5001], "size": 1048576,
"sha256": "..."},
      {"id": "reviews.csv_2", "nodes": [5002, 5003], "size": 1048507,
"sha256": "..."}
    ]
  }
}
```

HTTP API

Component	Endpoint	Purpose
Master	`POST /allocate`	Choose 2 nodes for each chunk of a new upload
Master	`POST /commit`	Record a file once its chunks are stored
Master	`GET /files` , `GET /files/<name>`	List files; return one file's chunk map with live copies
Master	`DELETE /files/<name>`	Delete a file and all its chunk copies
Master	`POST /heartbeat`	Receive a node's "I am alive" report every 3 s

Component	Endpoint	Purpose
Master	`GET /nodes`, `GET /events`	Node health; recent cluster events
Storage node	`PUT`, `GET`, `DELETE /chunks/<id>`	Save, return or delete one chunk
Storage node	`GET /chunks`, `GET /health`	List stored chunks; report status and disk use
Storage node	`POST /replicate`	Copy a chunk straight to another node (self-healing)

5. Implementation

The system is about 1,450 lines of Python in seven files, using only Flask, requests and matplotlib beyond the standard library.

File	Lines	Role
`config.py`	34	Shared settings: ports, chunk size, replication factor, heartbeat timings
`storage_node.py`	168	Flask storage node: save, return, list, delete and copy chunks; sends heartbeats
`master.py`	369	Flask master: metadata, chunk placement, failure detection, re-replication
`client.py`	306	Command-line client: upload, download, map, nodes, list, delete, events
`cluster.py`	139	Starts the master and 3 nodes from one terminal; stops and restarts nodes for the demo
`run_tests.py`	294	Runs tests T1–T6 plus one extra check, and times uploads and downloads
`charts.py`	136	Draws the three result charts

Tools. Python 3.10 or newer, Flask for the HTTP APIs, requests for HTTP calls between components, matplotlib for charts, Git and GitHub for version control, and VS Code as the editor. From the standard library: `json` (metadata file), `hashlib` (SHA-256), `threading` (heartbeats and the monitor), `pathlib` (chunk folders) and `argparse` (one port per node).

Key design decisions

- **The client moves the data.** Chunks travel straight between the client and the storage nodes. The master only answers "where" questions, so it never becomes a bandwidth bottleneck.
- **Least-loaded placement.** For each chunk the master picks the 2 live nodes holding the fewest copies, with ties going to the lower port. This is why test T6 shows a perfectly even spread.
- **Two-step upload.** The client first asks for a placement (`/allocate`), then stores the chunks, then reports what was stored (`/commit`). A file appears in the metadata only once its chunks are really on the nodes.
- **Checksums at every step.** A node rejects a chunk whose SHA-256 does not match the header sent with it. On download, the client checks every chunk and the whole file, and falls back to the other copy if a chunk is damaged.
- **Safe writes.** Chunks and the metadata file are written to a temporary file and then renamed, so a crash never leaves a half-written file.
- **Persistent metadata.** The chunk map is saved to `metadata.json` after every change, so a restarted master still knows every file.
- **Readable logs.** Each process prints one short line per meaningful event, such as `node 5002 is DEAD` or `re-replicated reviews.csv_0: node 5001 -> node 5003` , which makes the live demo easy to follow.

Client commands. `upload` , `download` , `list` , `map` (the chunk map as a grid of nodes), `nodes` (health and last heartbeat), `delete` and `events` (recent cluster events).

6. Fault tolerance and self-healing

A crashed node never interrupts reads: the client switches to the second copy at once, and the master restores two copies about 10 s later without anyone stepping in.

Reads during a failure. The client tries the copies of each chunk in turn, live nodes first. If a node refuses the connection, returns no chunk, or returns a chunk whose SHA-256 is wrong, the client moves to the next copy. No waiting for the master is needed, so the download in T3 finished 0.05 s after the crash.

Failure detection. Each storage node sends a heartbeat to the master every 3 s, carrying its port, chunk count and disk use. A monitor thread in the master checks every second; a node silent for more than 10 s is marked dead. Because the last heartbeat can come up to 3 s before the crash, detection lands 7 to 11 s after the crash. In T4 it came 10.0 s after the stop, 10.4 s after the last heartbeat.

Re-replication. In the same monitor pass, the master looks for chunks with fewer than 2 live copies:

1. It picks the least-loaded healthy node that does not already hold the chunk.
2. It tells a surviving holder to copy the chunk there (`POST /replicate`). The copy goes node to node, never through the master.
3. When the copy succeeds, the master replaces the dead node with the new one in the chunk map and saves the metadata.
4. When every chunk has 2 live copies again, it logs `cluster healthy` .

In T4 the three lost copies were re-created 0.04 s after detection.

When no healthy node is left. If only one node is alive (T5), a chunk cannot get a second copy. The master keeps the dead node in the chunk map and logs that it is waiting. If that node returns with its data, its copies count again at once, with no copying needed.

Rejoin and cleanup. A node that returns after being marked dead may still hold copies that were re-created elsewhere while it was away. The master asks it for its chunk list and deletes those stale copies. In the extra test E1, node 5002 returned and its 3 stale copies were removed.

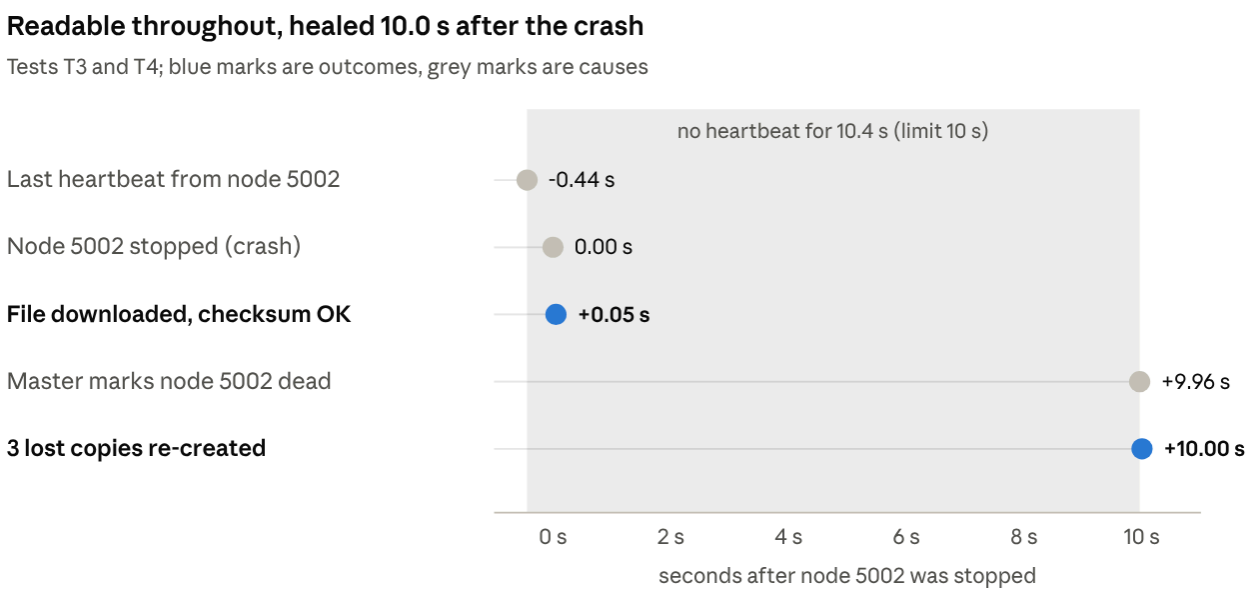
7. Testing and results

All 7 tests passed: the six roadmap tests T1–T6 and one extra rejoin check, E1. The key test, T3, downloaded the full file with one node stopped and a checksum identical to the original.

The tests are automated in `run_tests.py` . It starts a fresh cluster in its own folder, uses random files with known SHA-256 values, stops nodes by killing their processes (a real crash, with no goodbye message), and checks both the master's chunk map and what each node really holds on disk.

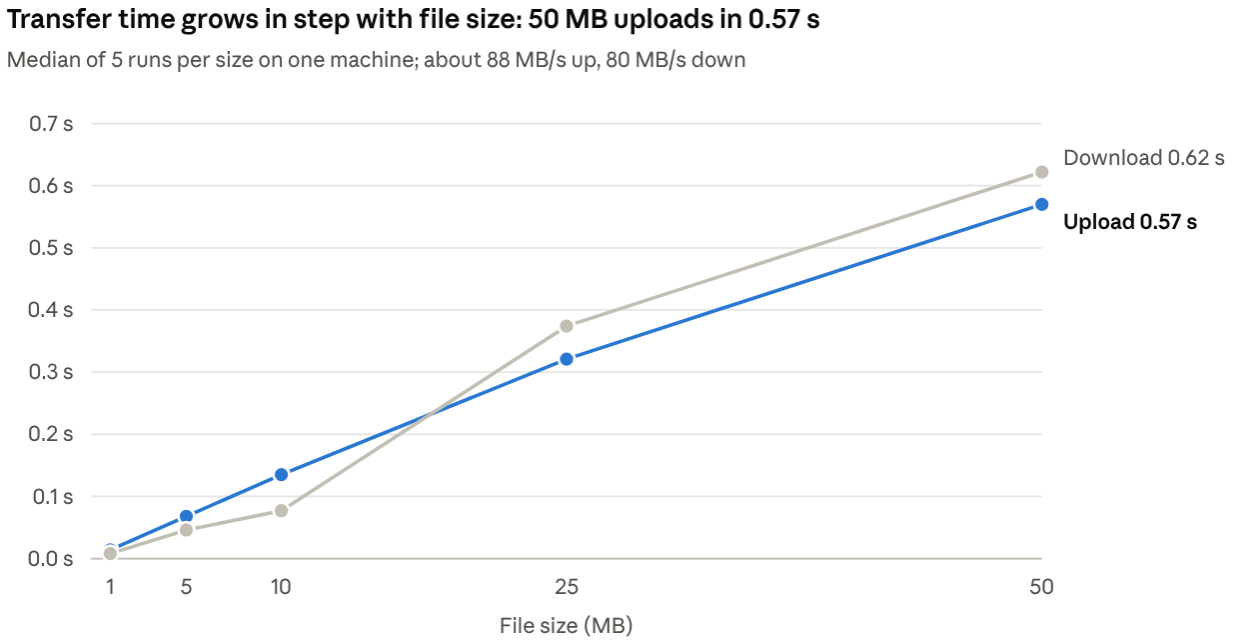
#	Test	Expected	Observed	Result
T1	Upload a 5 MB file	5 chunks, each stored on 2 different nodes	5 chunks, 10 copies, each pair on different nodes (5001: 4, 5002: 3, 5003: 3)	Pass
T2	Download with all nodes running	SHA-256 checksum matches the original	Checksum matches; 0.04 s	Pass

#	Test	Expected	Observed	Result
T3	Stop one storage node, then download	File downloads complete; checksum matches	Node 5002 stopped; file complete in 0.05 s, 1 of 5 chunks read from its second copy; checksum matches	Pass
T4	Keep the node stopped for 10+ seconds	Master marks it dead and copies its chunks to a healthy node	Marked dead after 10.4 s without a heartbeat; its 3 copies re-created on healthy nodes; every chunk back to 2 live copies	Pass
T5	Stop a second node after re-replication	File still downloads correctly	Nodes 5002 and 5003 both down; all 5 chunks read from node 5001; checksum matches	Pass
T6	Upload 10 files of different sizes	Chunks spread roughly evenly across the 3 nodes	10 files of 0.3 to 10 MB, 45 chunks, 90 copies: exactly 30 on each node; all 10 files verified	Pass
E1	Restart both stopped nodes (extra)	Nodes rejoin; 2 live copies again; stale copies removed	Both rejoined; every chunk has 2 live copies; 3 stale copies deleted from node 5002	Pass



run_tests.py · master event log during T3 and T4

The download in T3 needed no help from the master: it switched to the second copy at once. Healing waited for the 10 s silence limit, then took 0.04 s.



run_tests.py · median of 5 runs per file size, 1 to 50 MB

Times rise roughly in a straight line with file size, as expected when every 1 MB chunk costs about the same. These figures come from the test machine (Python 3.13 on Linux); running `python run_tests.py` on the demo laptop records that laptop's own numbers and charts.

8. Discussion and limitations

The results answer the problem statement: a file stays downloadable, complete and unchanged after a storage node stops, and even after two of the three stop.

What the results show

- **Replication works because placement is strict.** Every chunk had its two copies on different nodes, so the loss of any one node always left one copy. T5 shows the limit of a factor of 2: after healing, a second failure still left one full copy of the file.
- **Healing is fast; detection is the slow part.** Copying the lost chunks took 0.04 s, but deciding that the node was dead took about 10 s. That wait is a deliberate trade-off: a shorter timeout heals sooner but risks copying data for a node that was only briefly slow.
- **Checksums make failures safe.** Each chunk and the whole file are verified, so a damaged or missing copy is skipped rather than returned. All downloads in all tests matched the original SHA-256.

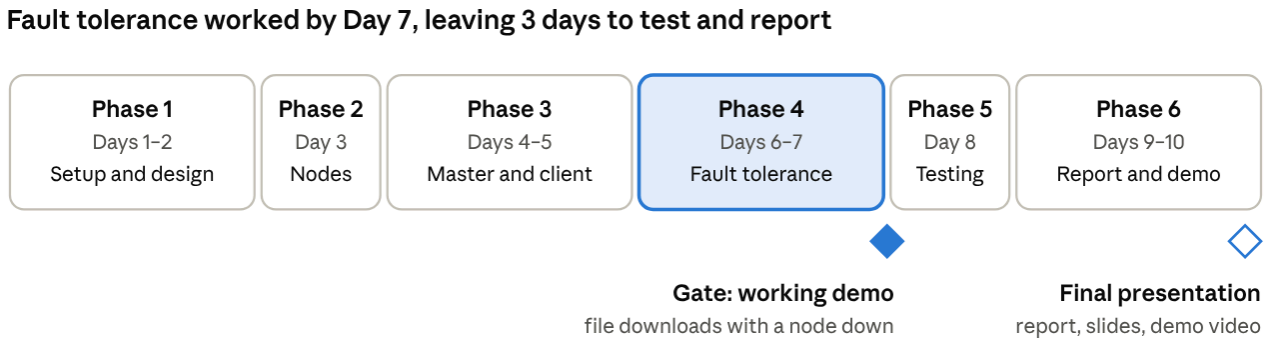
- **Least-loaded placement balances storage.** Placing each chunk on the two emptiest nodes gave an exact 30 / 30 / 30 split in T6.
- **Transfer time grows in step with file size.** A 50 MB file uploaded in 0.57 s and downloaded in 0.62 s, about 85 MB/s. Uploads write every byte twice, yet stayed close to download speed on a local machine.

Known limitations

- **The master is a single point of failure.** If it stops, no new uploads or downloads can start, although the chunk map survives in `metadata.json`. Real HDFS adds a standby NameNode.
- **All nodes run on one laptop.** Real network delays, bandwidth limits and partitions are not tested, so the timings are a best case.
- **Files cannot be edited after upload, only replaced,** as in HDFS's write-once model.
- **Three nodes and a factor of 2 cap resilience.** Two simultaneous failures make any chunk stored only on those two nodes unavailable until one returns.
- **No security.** There are no user accounts, no encryption and no access control, as agreed in the scope.

9. Project management

The work ran in six phases over 10 days, with a working fault-tolerance demo as the gate at the end of Day 7.



10-day roadmap · 6 phases, 1 gate (from the project roadmap)

Keeping the last three days free of new features left time to run every test, write this report and rehearse the demo twice.

Phase	Days	Main tasks	Output
1. Setup and design	1–2	Repository, tools, chunk size, metadata format and API endpoints	Design notes, empty project
2. Storage nodes	3	Flask node that saves, returns and lists chunks; `/health` endpoint	`storage_node.py`
3. Master, upload and download	4–5	Metadata, 2 nodes per chunk, client split and rebuild, checksums	`master.py`, `client.py`
4. Fault tolerance	6–7	Heartbeats every 3 s, dead after 10 s, reads from the surviving copy, re-replication	Heartbeat and re-replication code
5. Testing and analytics	8	Tests T1–T6, timings, charts	`run_tests.py`, results, charts
6. Report and presentation	9–10	Report, slides, backup demo video, two rehearsals	Report, slides

Team roles. Four roles cover the work; a smaller group combines roles 3 and 4.

Role	Member	Main responsibilities	Key phases
Storage node developer	(add name)	Storage node API, chunk folders, health endpoint	2
Master and client developer	(add name)	Metadata, chunk placement, upload and download, checksums	3
Fault tolerance and testing lead	(add name)	Heartbeats, failure detection, re-replication, test cases, charts	4, 5
Documentation and presentation lead	(add name)	Report, slides, demo video, rehearsal schedule	1, 6

The group met for 15 minutes each day, and all code went through GitHub.

10. Conclusion and future work

The project met all five objectives: files are partitioned into chunks, every chunk is replicated on two nodes, downloads survive node failures with matching checksums, failed nodes are detected within about 10 s, and lost copies are re-created automatically. The same ideas that let HDFS store petabytes on unreliable machines work, at small scale, in about 1,450 lines of Python.

Future work

- **Standby master:** copy the metadata to a second master that takes over when the first stops, removing the single point of failure.
- **Multi-machine deployment:** run nodes on separate computers to measure real network delays.
- **Rack-aware placement:** keep the two copies of a chunk in different racks or rooms, as HDFS does, so one power or network fault cannot remove both.
- **Background checksum scans:** have nodes re-check stored chunks regularly and replace damaged copies before anyone reads them.
- **A web dashboard:** a small Django page showing nodes, files and the live chunk map, for monitoring and demos.
- **Security:** user accounts, access control and encrypted transfers.

References

1. Apache Software Foundation. HDFS Architecture. Apache Hadoop documentation.
2. Apache Software Foundation. hdfs-default.xml (default values for replication, block size and heartbeat interval). Apache Hadoop 2.10.2 documentation.
3. Shvachko, K., Kuang, H., Radia, S. and Chansler, R. (2010). The Hadoop Distributed File System. *IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)*.
4. Ghemawat, S., Gobioff, H. and Leung, S.-T. (2003). The Google File System. *ACM Symposium on Operating Systems Principles (SOSP)*.
5. Pallets Projects. Flask documentation. https://flask.palletsprojects.com

Appendix A. How to run the project

Install Python 3.10 or newer, then from the project folder:

```
pip install -r requirements.txt
python cluster.py --fresh                       # master + 3 storage nodes
python client.py upload samples/reviews.csv     # in a second terminal
python client.py map reviews.csv
python client.py download reviews.csv
python run_tests.py                             # tests T1-T6, E1 and charts
```

In the `cluster.py` terminal, type `stop 5002` to crash a node, `start 5002` to bring it back, and `quit` to stop everything. The README in the repository lists every command.

Appendix B. Live demo script (about 3 minutes)

1. Start the cluster with `python cluster.py --fresh`; the three nodes join the master.
2. Upload `samples/reviews.csv` (3 MB): the client shows 3 chunks, each on 2 nodes.
3. Show the chunk map with `python client.py map reviews.csv`.
4. Type `stop 5002` to crash a storage node.
5. Download the file: it completes, prints `Checksum OK`, and marks the chunk read from its backup copy.
6. Wait about 10 seconds: the master logs `node 5002 is DEAD`, then the re-replication lines.
7. Show the map again: every chunk has 2/2 live copies and none sits on node 5002.
8. Type `start 5002`: the node rejoins and the master deletes its stale copies.